

Problem1: Cache (31 points)

Consider a **16-bit machine** with a **2-way** set associative cache. The size of each block is **8 bytes**, and the total number of sets is 2. **LRU** policy is used for eviction and **write back** policy is used. The following table shows the content of the data cache at time T.

Set	Tag	Valid	Byte0	Byte1	Byte2	Byte3	Byte4	Byte5	Byte6	Byte7
0	0x0C0	1	0x2F	0x54	0x30	0xCC	0x66	0x9D	0xBC	0x7B
0	0xF0E	0	0xEE	0xD6	0xAC	0xAC	0xD0	0x0A	0xAC	0x9F
1	0x140	0	0x4C	0xAB	0xCD	0xEF	0xAD	0xBE	0xFF	0xEF
1	0x1E0	1	0x4B	0x19	0x24	0x25	0x99	0xDD	0x24	0xAF

1. What is the total size of the cache? (3')

$2 * 2 * 8 = 32 \text{ bytes}$

2. How would a 16-bit physical memory address be split into tag/set-index/block-offset fields in this machine? (6')

tag 12 bits, set-index 1 bits, and block-offset 3 bits

3. A short program will read memory in the following sequences starting from time T. Each access will read **one byte**. Please fill the following blanks. If there is a cache miss, fill '-' for 'Byte Returned'. (10')

Order	Address	Hit/Miss	Byte Returned
1	0xF0E2	M	-
2	0x0C08	M	-
3	0x0C03	H	0xCC
4	0x2C3B	M	-
5	0x1E0A	M	0x24

One of the TA writes a program and tests it on this cache. The size of short type is **2 bytes**. The cache is **empty** before the execution of the program. Please only consider **data cache** access and ignore instruction cache access. Other processes will not interfere the status of memory and cache during this execution.

```
1. #define N 8
2. short A[N];
3. short B[N];
4. short C[N];
5.
6. void dot_product() {
7.     int i;
8.     for (i = 0; i < N; i++) {
9.         C[i] = A[i] * B[i] + C[i];
10.    }
11. }
```

Assume that the address of A[0] is 0xa00. The address of B[0] is 0xb00. The address of C[0] is 0xc00. Answer the following questions:

4. Please calculate **the cache miss rate**. (3')

$3/4=75\%$

5. We can modify the cache through one of the following ways:

- a) Double the number of sets while keeping the cache size unchanged.
- b) Double the block size while keeping the cache size unchanged.
- c) Increase the cache size via doubling the number of cache lines in each set.

Please choose the option(s) above that can optimize the miss rate of the original program and calculate the miss rate respectively.(6')

$C\ 6/32=18.75\%$

6. Besides modifying the cache hardware parameters, can you propose a software-based solution to improve the cache hit rate? (3')

(Hint: Consider adjusting the starting addresses of arrays.)

Move the arrays in memory to ensure at least one of A[0], B[0], or C[0] lands in set0, and at least one in set1

e.g. Move C to 0xC08

(The answer is not unique.)